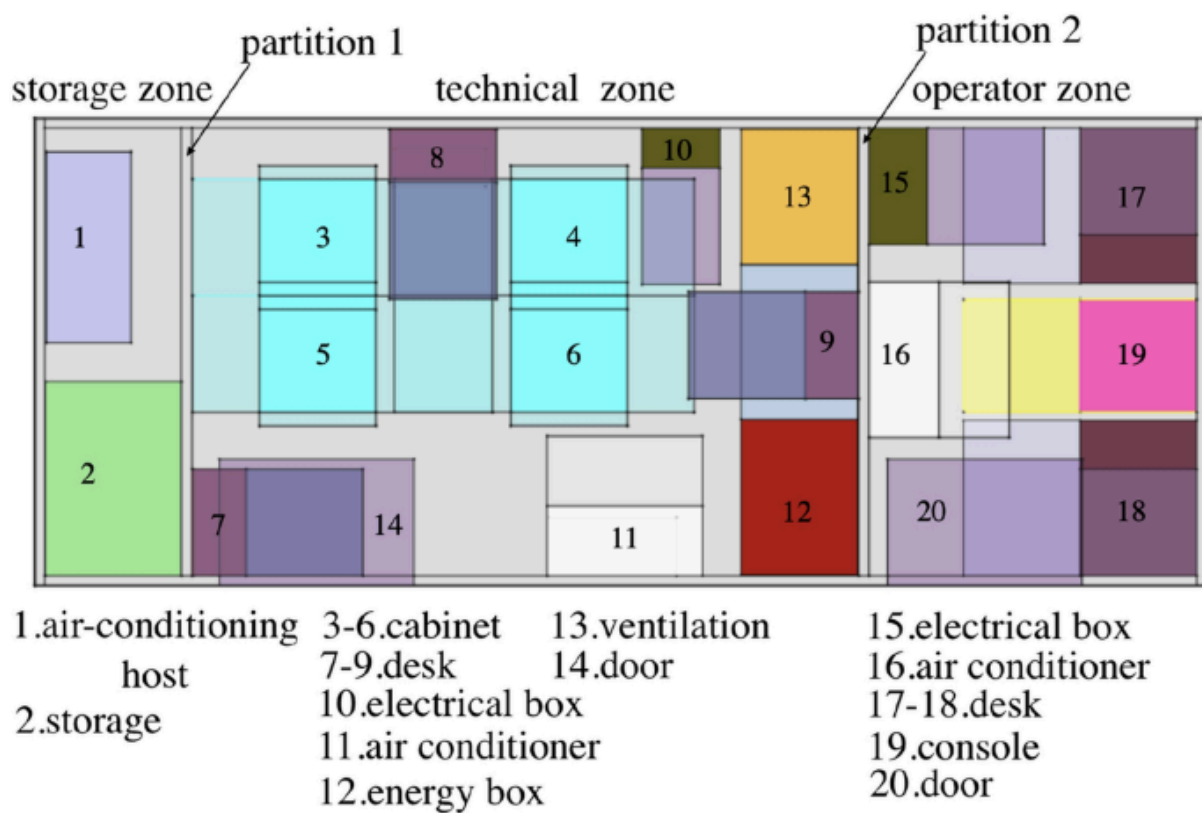




Exercices avancés de Programmation Parallèle et Distribuée

Optimisation du Floorplan

Notre deuxième étude de cas illustre un problème hautement irrégulier et symbolique. La solution que nous développons intègre un algorithme d'ordonnancement des tâches.



Exemple illustratif du problème ¹ de l'Optimisation du Floorplan avec contraintes fonctionnelles

Contexte du Floorplan

La VLSI (Very Large Scale Integration) est un processus utilisé pour concevoir des composants électroniques tels que les microprocesseurs et les puces mémoire, qui contiennent des millions de transistors. La conception des composants VLSI est un processus computationnellement exigeant. Les ordinateurs sont largement utilisés pour vérifier la correction d'un circuit, l'implanter dans une zone bidimensionnelle et

¹ Song, X., Poirson, E., Ravaut, Y. *et al.* Multi-objective optimization of layout with functional constraints. *Optim Eng* 24, 1849–1882 (2023). <https://doi.org/10.1007/s11081-022-09754-z>



Exercices avancés de Programmation Parallèle et Distribuée

généraliser les modèles nécessaires aux tests après fabrication. Beaucoup de ces problèmes impliquent une recherche exhaustive ou guidée par heuristique sur un grand espace de solutions possibles.

Nous nous intéressons ici à un problème de mise en page. La première étape du processus de conception VLSI produit généralement un ensemble de blocs rectangulaires indivisibles appelés cellules. Lors d'une deuxième étape, les informations d'interconnexion sont utilisées pour déterminer le positionnement relatif de ces cellules. La troisième étape consiste à choisir des implémentations pour les différentes cellules afin d'optimiser la surface totale. C'est cette troisième étape, l'optimisation du floorplan, que nous allons traiter avec un algorithme parallèle. Cette étape est cruciale dans le processus de conception, car le coût d'une puce dépend généralement de sa surface.

L'optimisation du floorplan VLSI peut être expliquée par analogie avec la conception d'une cuisine. Supposons que nous ayons déterminé les composants d'une cuisine (équivalent de l'étape 1 en VLSI) et leur agencement (étape 2). Par exemple, nous voulons une cuisinière, un réfrigérateur, une table et un évier, avec la contrainte que la cuisinière soit à côté du réfrigérateur et que la table soit proche de l'évier. Supposons aussi que nous puissions choisir parmi plusieurs modèles pour chacun de ces composants, avec des formes différentes mais une même surface au sol. L'optimisation du floorplan consiste alors à choisir les modèles qui utilisent au mieux l'espace disponible.

En VLSI, un floorplan est représenté par une paire de graphes polaires, conventionnellement appelés graphe vertical et graphe horizontal. (Un graphe polaire est un graphe orienté acyclique avec une unique source et un unique puits. Le terme orienté signifie que les arêtes ont une direction, et acyclique qu'il n'y a pas de cycles.) Ces graphes définissent quelles cellules sont adjacentes verticalement et horizontalement. Chaque arc représente une cellule, et les nœuds (sauf la source et le puits) relient les cellules qui doivent avoir des bords adjacents.

Bien qu'une cellule ait une surface fixe, elle peut avoir plusieurs implémentations avec différents rapports d'aspect. Si nous avons N cellules et que la cellule i possède m_i implémentations, alors le nombre total de configurations possibles du floorplan est :

$$\prod_{i=0}^{N-1} I(c_i).$$

Par exemple, la Figure 2.27 illustre un problème d'optimisation de floorplan avec trois cellules et six configurations possibles.



Exercices avancés de Programmation
Parallèle et Distribuée

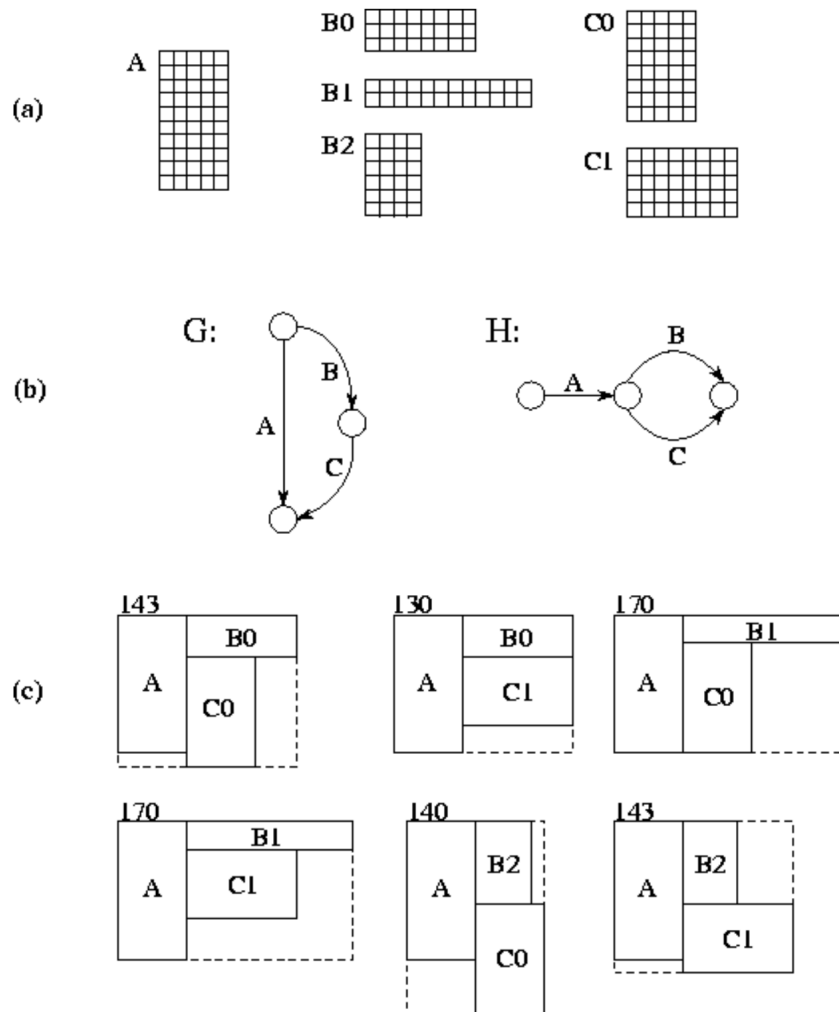


Figure 2.27: A floorplan optimization problem. The three cells A , B , and C , have 1, 3, and 2 implementations each, respectively. In (a) are the alternative implementations. In (b) are the $\mathcal{G}$ and $\mathcal{H}$ graphs, which state that B must be above C , and that A must be to the left of B and C , respectively. In (c) are the alternative floorplans that satisfy the constraints; each is labeled with its area. The lowest area floorplan is constructed from A , B_0 , and C_1 and has an area of 130.



Exercices avancés de Programmation Parallèle et Distribuée

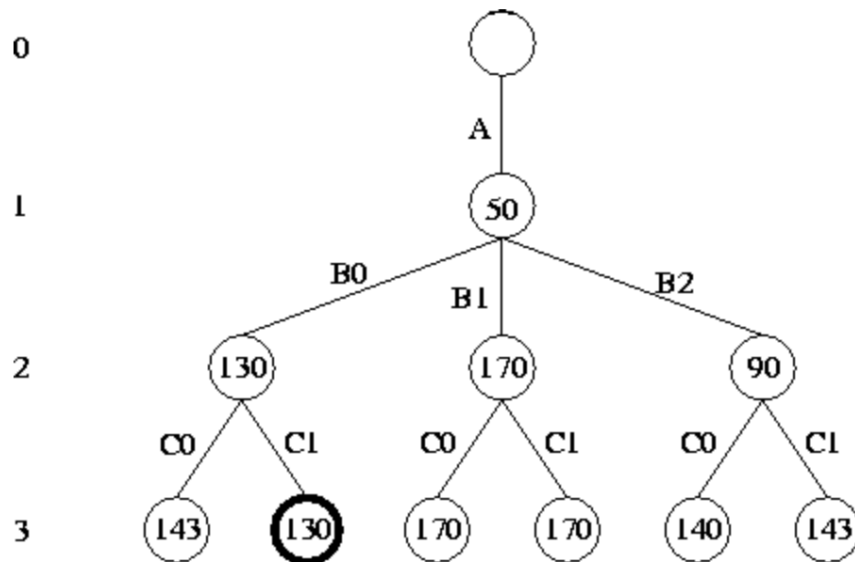


Figure 2.28: Solving a floorplan optimization problem. This is the search tree corresponding to the problem illustrated in Figure 2.27. Level 0 is the root. At level 1, an implementation has been chosen for A; the three level 2 subtrees represent the choices for B and the level 3 leaves the choices for C. The number in each tree node represents the area of the associated (partial) solution. The optimal configuration is (A,B0,C1) and has area 130.

L'objectif est d'identifier la configuration avec la plus petite surface, où la surface est définie comme le produit des dimensions maximales en largeur et en hauteur. Cette recherche peut être réalisée en explorant un arbre de recherche représentant toutes les configurations possibles. Comme illustré dans la Figure 2.28, le niveau i de cet arbre correspond à la sélection d'une implémentation pour i cellules.

Un algorithme de recherche exhaustif peut être utilisé pour parcourir tout l'arbre, mais cette stratégie devient vite irréalisable dès que le nombre de cellules augmente. Par exemple, un problème avec seulement 20 cellules et 6 implémentations par cellule génère un espace de recherche de taille :

$$6^{20} \approx 4 \times 10^{15}$$

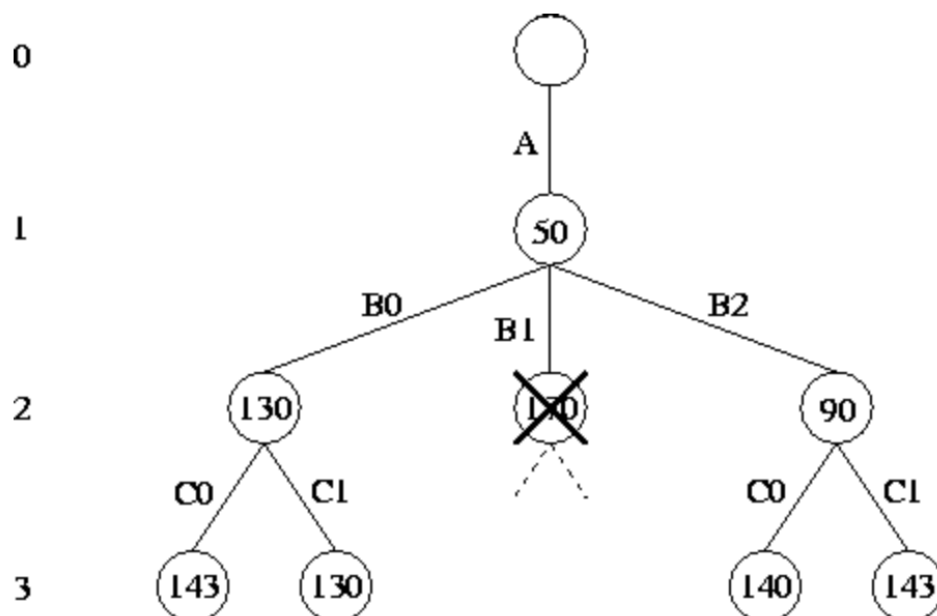
Heureusement, nous pouvons considérablement réduire le nombre de nœuds explorés en utilisant la technique de recherche avec élagage par borne (branch-and-bound). L'idée est de garder une trace de la meilleure solution trouvée jusqu'à présent (avec la plus petite surface). Avant d'explorer un nœud, nous vérifions si la surface de la configuration partielle est déjà plus grande que celle de la meilleure solution



Exercices avancés de Programmation Parallèle et Distribuée

connue. Si c'est le cas, nous savons que ce nœud ne peut pas conduire à une meilleure solution et nous pouvons abandonner tout son sous-arbre (élagage).

La Figure ci-dessous illustre cette approche : le sous-arbre d'un nœud de niveau 2 est supprimé car son coût (170) est supérieur à celui de la meilleure solution trouvée (130).



Conception de l'Algorithme du Floorplan

Partitionnement

L'algorithme de recherche avec élagage (Algorithm 2.2) ne présente pas de structure de données évidente pour appliquer des techniques de décomposition par domaine. Nous utilisons donc une décomposition fonctionnelle fine, où chaque nœud de l'arbre de recherche est exploré par une tâche distincte.

L'exploration de l'arbre génère donc un front d'onde de nouvelles tâches qui s'exécutent en parallèle, mais seules les tâches situées sur ce front peuvent s'exécuter simultanément. Nous devons également gérer la mise à jour de la borne globale A, qui est utilisée pour stocker la meilleure solution trouvée.

Un problème de conception apparaît ici : une exploration en largeur d'abord (breadth-first) risque de retarder la découverte des bonnes solutions et de limiter l'élagage, entraînant des calculs redondants. Ce problème doit être pris en compte dans les phases suivantes.



Exercices avancés de Programmation Parallèle et Distribuée

Communication

Dans une recherche simple (Algorithm 1.1), les tâches exécutent leur travail de manière indépendante et ne communiquent que pour signaler une solution. En revanche, l'algorithme branch-and-bound nécessite une communication pour mettre à jour et consulter la borne A.

Une approche consiste à confier la gestion de A à une tâche centralisée, avec laquelle toutes les autres tâches communiquent lorsqu'elles trouvent une solution. Cette approche est simple et peut être efficace si le coût de la communication est faible. Toutefois, elle ne passe pas à l'échelle (non scalable) lorsque le nombre de tâches augmente.

Une alternative consiste à partager la responsabilité de A entre plusieurs gestionnaires locaux, qui échangent périodiquement leurs meilleures solutions trouvées.

Agglomération

À ce stade, nous devons prendre en compte les performances réelles sur l'ordinateur cible. Nous pouvons réduire le nombre de tâches fines en regroupant plusieurs nœuds dans une même tâche. Par exemple, au lieu de créer une tâche pour chaque appel récursif de recherche, nous pouvons basculer vers une recherche séquentielle en profondeur (depth-first) après un certain niveau dans l'arbre.

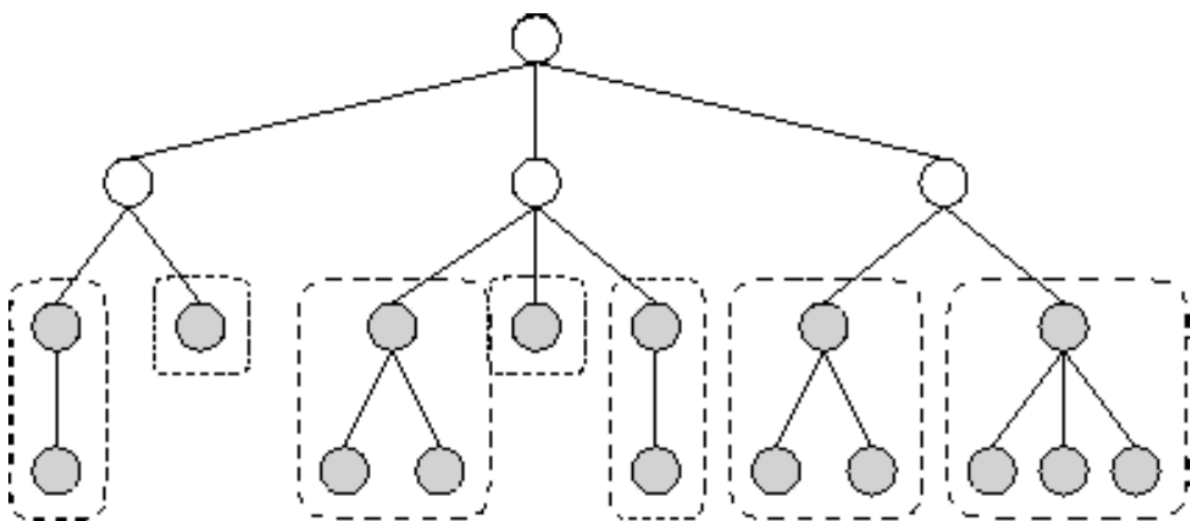


Figure 2.30: Increasing granularity in a search problem. In this figure, we agglomerate by switching to a sequential search at level two in the search tree. A task is created for each subtree rooted at level two.



Exercices avancés de Programmation Parallèle et Distribuée

Affectation des tâches (Mapping)

Nous devons mettre en place une stratégie d'ordonnancement des tâches efficace :

- Une approche consiste à allouer des sous-arbres entiers à des travailleurs (workers) qui les explorent en profondeur pour maximiser l'élagage.
 - Une autre stratégie consiste à attribuer dynamiquement les tâches aux processeurs disponibles pour équilibrer la charge de travail.
 - Un modèle décentralisé permet aux travailleurs de s'échanger des tâches en fonction de leur disponibilité.
-

```
procedure b&b_search( $A$  )
begin
   $A_{min} = \infty$ 
  b&b_search_1( $A$  )
end

procedure b&b_search_1( $A$  )
begin
  score = eval( $A$  )
  if(score <  $A_{min}$ ) then
    if (leaf( $A$  )) then
       $A_{min} = \text{score}$ 
      report solution and score
    else
      foreach child  $A_i$  of  $A$ 
        b&b_search_1( $A_i$ )
      endfor
    endif
  endif
end
```

Algorithm 2.2 Branch-and-bound search is similar to Algorithm 1.1 (simple search), but uses a search bound A_{min} to prune the search.



Exercices avancés de Programmation
Parallèle et Distribuée

Questions

1. Qu'est-ce que l'optimisation du floorplan en VLSI et pourquoi est-elle importante ?
2. Comment peut-on comparer le problème du floorplan à la conception d'une cuisine ?
3. Quels sont les principaux défis liés à l'optimisation du floorplan ?
4. Comment les graphes polaires sont-ils utilisés pour représenter le floorplan ?
5. Quels sont les paramètres clés qui influencent le nombre total de configurations possibles d'un floorplan ?
6. Pourquoi est-il important de minimiser l'aire totale d'un floorplan ?
7. Comment fonctionne l'algorithme de recherche exhaustive pour trouver la meilleure configuration ?
8. Pourquoi l'algorithme de recherche exhaustive est-il inefficace pour les problèmes de grande taille ?
9. Comment l'algorithme de branch-and-bound améliore-t-il la recherche de la solution optimale ?
10. Quel rôle joue l'élagage (pruning) dans l'algorithme de branch-and-bound ?
11. Quels sont les défis liés à la parallélisation de la recherche d'un floorplan optimal ?
12. Pourquoi une stratégie de recherche en largeur (breadth-first) peut-elle réduire l'efficacité du pruning ?
13. Quelles stratégies permettent de mieux gérer la variable globale A (qui stocke la meilleure solution trouvée) en environnement parallèle ?
14. Quels sont les avantages et inconvénients d'une gestion centralisée vs décentralisée de la recherche de solutions ?
15. Comment la granularité des tâches affecte-t-elle l'efficacité de l'algorithme parallèle ? Pourquoi est-il utile de passer d'une recherche parallèle à une recherche séquentielle à partir d'une certaine profondeur dans l'arbre de recherche ?
16. Comment un algorithme de scheduling des tâches peut-il améliorer la performance globale ?
17. Quelles sont les approches possibles pour équilibrer la charge de travail entre les processeurs ?
18. Comment la communication entre les tâches influence-t-elle la performance de l'algorithme parallèle ?
19. Quels seraient les impacts de cette optimisation dans d'autres domaines que le VLSI (ex : logistique, urbanisme, architecture) ?
20. Comment pourrait-on améliorer davantage l'algorithme pour traiter des instances encore plus grandes ?
21. Existe-t-il des techniques d'apprentissage automatique qui pourraient être appliquées à ce problème ?



Dr B. Diop
CINERI, 2025

Exercices avancés de Programmation
Parallèle et Distribuée
